

Deep Learning

Lecture 9: Generalisation Theory

Chris G. Willcocks

Durham University



1 Foundations

- universal approximation theorems
- empirical risk minimisation
- noise, training set size, and regularisation

2 Statistical Learning Theory

- VC dimension and generalisation bound
- Rademacher complexity
- why supervised learning is “solved”

3 Unsupervised Learning Theory

- the objective mismatch
- Kolmogorov complexity
- no free lunch theorem and Occam’s razor
- NN SGD approximates program search

4 Scaling Laws and Modelling Theory

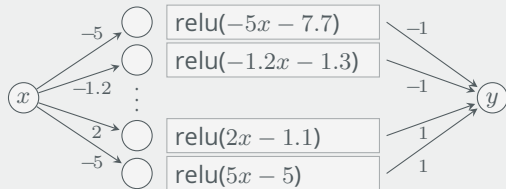
- bias-variance tradeoff on nasty densities
- double descent theory and scaling laws
- the eval-reality gap vs human learning



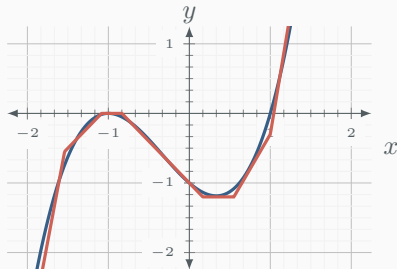
Theorem: universal function approximator

Arbitrary width

A network with a single hidden layer, containing a finite number of neurons, can approximate any continuous function under mild assumptions.



original function $f(x)=x^3+x^2-x-1$



$$f(x) = -r(-5x - 7.7) - r(-1.2x - 1.3) - r(1.2x + 1) + r(1.2x - 0.2) + r(2x - 1.1) + r(5x - 5)$$

Theorem: universal function approximator

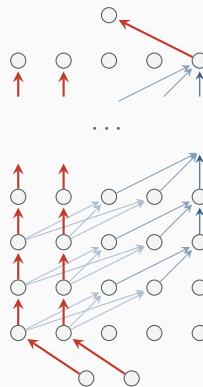
Arbitrary depth (fixed width)

Does the theorem still hold for fixed width and arbitrary depth? **Yes!**

For a network of n inputs and m outputs, [1] show universal approximation holds true for:

- width $n + m + 2$ for almost any activation function
- width $n + m + 1$ for most activation functions

Short YouTube visual proof 





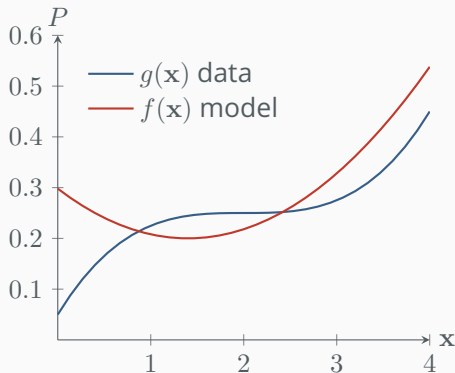
Finding a model

Lets illustrate this in 1D, but try to imagine it in ND . Given the target probability distribution of our data

$$g(\mathbf{x}) = P(X, Y)$$

we want to find (design) a model $f(\mathbf{x}; \theta)$ with parameters θ and optimise θ such that

$$f(\mathbf{x}; \theta) = g(\mathbf{x})$$



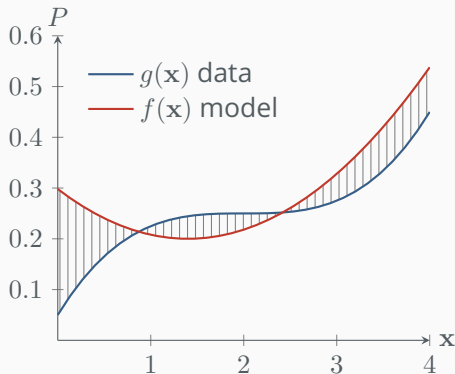
Optimising the model

The total error is therefore the entire area.
Modifying the parameters θ will cause the area to change, where we want to find

$$\hat{\theta} = \arg \min_{\theta} \int \mathcal{L}(f(\mathbf{x}; \theta), g(\mathbf{x})) d\mathbf{x}.$$

where $\mathcal{L}$ is a 'loss function', e.g. a 0-1 loss function $\mathcal{L}(\hat{x}, x) = \mathbb{I}(\hat{x} \neq x)$ or a mean squared error loss.

The solution is a function f that has the capacity to exactly represent $g(\mathbf{x})$





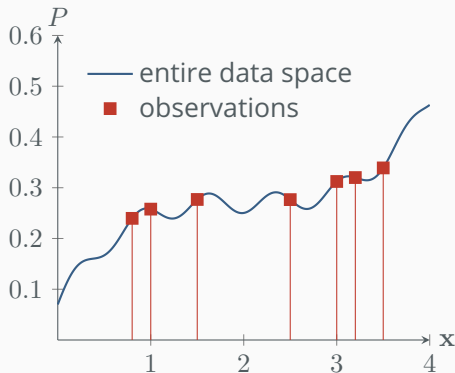
The generalisation problem

However there's a big problem:

In practice, we can't observe all of $g(\mathbf{x})$

This means:

1. We don't know how smooth the function is between the observations
2. Noise can be difficult to interpret
3. Optimisation is highly sensitive to the sampling process



Definition: true and empirical risk

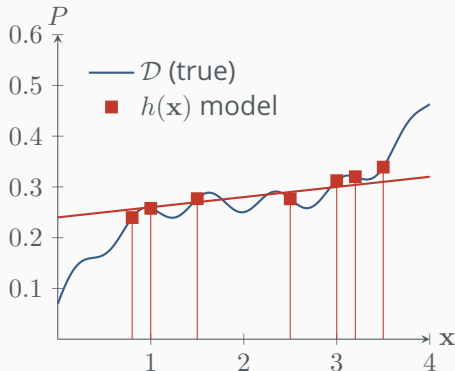
The **true risk** (expected error) averages over the entire data distribution $\mathcal{D}$:

$$R(h) = \mathbb{E}_{(\mathbf{x}, y) \sim \mathcal{D}} [\mathcal{L}(h(\mathbf{x}), y)]$$

Since we can't compute this, we minimise the **empirical risk** over N samples:

$$\hat{R}(h) = \frac{1}{N} \sum_{i=1}^N \mathcal{L}(h(\mathbf{x}_i), y_i)$$

Key question: When does $\hat{R}(h) \approx R(h)$?

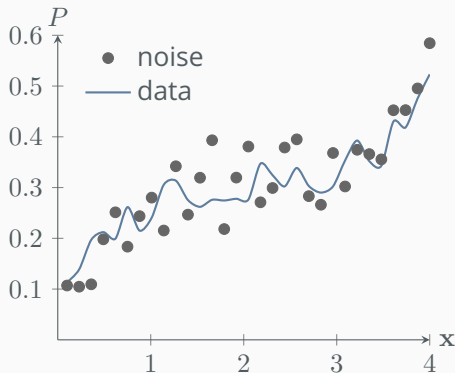




Noise and regularisation

Given that the shape of the distribution outside of the observations is unknown, it is easy to overfit to noise, especially when you have limited data.

We can often regularise our function to be invariant to this.

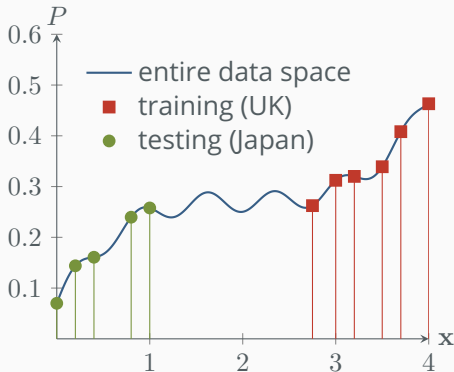




Out-of-distribution data

Usually the training dataset collection process draws samples from the data space in a way that is not **independent and identically distributed** (abbreviated i.i.d.) to the expected testing (operational) conditions of the model.

Sampling data in a way that is representative of the task/testing/operational distribution is extraordinarily difficult to do properly. It is often a worthwhile investment though!





Similarity measures

How can we evaluate the generalisation of unsupervised generative models?

Deep pre-trained network activations work surprisingly well (e.g. FID, LPIPS [2] for perceptual similarity between images):

```
import lpips
distance_fn = lpips.LPIPS(net="alex")
```

Example code usage [↗](#)



Model samples (leftmost column) and nearest neighbours in training set (LPIPS distance [2]) increasing to right.



Bias amplification

Judging models solely based on pre-trained measures such as FID can amplify the inherited biases.

FID (currently how the majority of image-based generative model papers are ranked) can favour less diversity (with focus shifted towards the distribution mode).



Temperature →

Sampling away from the mode of the data distribution



Core Definitions

- **Function class** $\mathcal{F}$: all functions our architecture can represent
- **Hypothesis** $h \in \mathcal{F}$: a specific learned function
- **Data distribution** $\mathcal{D}$: the true (unknown) distribution over $(\mathbf{x}, y)$

Using notation from ERM: $R(h)$ is the true risk over $\mathcal{D}$, and $\hat{R}(h)$ is the empirical risk over our N samples.

Regret

How suboptimal is our learned hypothesis?

$$\text{Regret}(h) = \overbrace{R(h)}^{\text{our error}} - \overbrace{\inf_{h^* \in \mathcal{F}} R(h^*)}^{\text{best possible in } \mathcal{F}}$$

Low regret $\Leftrightarrow$ we extracted all “value” from the data.

Key question: How does $\hat{R}(h) \rightarrow R(h)$ as $N \rightarrow \infty$?



Definition: Shattering

A function class $\mathcal{F}$ **shatters** a set of points $\{x_1, \dots, x_n\}$ if it can realise all 2^n possible binary labellings.

Vapnik–Chervonenkis (VC) Dimension

$$d_{\text{VC}}(\mathcal{F}) = \max\{n : \mathcal{F} \text{ can shatter } n \text{ points}\}$$

Measures the “capacity” or “expressiveness” of $\mathcal{F}$.

Examples

- Linear classifiers in $\mathbb{R}^d$: $d_{\text{VC}} = d + 1$
- Axis-aligned rectangles in $\mathbb{R}^2$: $d_{\text{VC}} = 4$
- $\sin(\omega x)$ for $\omega \in \mathbb{R}$: $d_{\text{VC}} = \infty$

Historical note: VC theory was developed partly to handle parameters with *infinite precision*. In practice, float32 already constrains effective capacity.



Theorem: VC Generalisation Bound (Uniform Convergence)

For function class $\mathcal{F}$ with VC dimension d_{VC} , with probability $\geq 1 - \delta$ over the draw of N i.i.d. training samples, **simultaneously for all** $h \in \mathcal{F}$:

$$\left| \underbrace{R(h)}_{\text{true risk}} - \underbrace{\hat{R}(h)}_{\text{train error}} \right| \leq \underbrace{O \left(\sqrt{\frac{d_{\text{VC}} \log N + \log(1/\delta)}{N}} \right)}_{\text{generalisation gap}}$$

Interpretation:

- More data $N \Rightarrow$ gap shrinks as $O(1/\sqrt{N})$
- Higher capacity $d_{\text{VC}} \Rightarrow$ larger gap
- Holds *uniformly* over all $h \in \mathcal{F}$

Key insight: Train error $\approx$ test error (with high probability) when $N \gg d_{\text{VC}}$.

$\Rightarrow$ We can confidently collect large datasets knowing models *must* generalise.



Definition: Empirical Rademacher Complexity

For function class $\mathcal{F}$ and sample $S = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$:

$$\hat{\mathcal{R}}_S(\mathcal{F}) = \mathbb{E}_{\sigma} \left[\sup_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^N \sigma_i f(\mathbf{x}_i) \right]$$

where $\sigma_i \stackrel{\text{iid}}{\sim} \text{Uniform}\{-1, +1\}$.

Measures how well $\mathcal{F}$ can correlate with random noise.

Theorem: Rademacher Bound

With probability $\geq 1 - \delta$, for all $h \in \mathcal{F}$:

$$\left| R(h) - \hat{R}(h) \right| \leq 2 \mathcal{R}_N(\mathcal{F}) + 3 \sqrt{\frac{\log(2/\delta)}{2N}}$$

Advantages over VC:

- **Data-dependent** $\Rightarrow$ tighter in practice
- Works for real-valued (not just binary) functions
- Connects naturally to margin-based bounds



The Key Insight

These bounds tell us **exactly** the generalisation gap.

- We know how much data we need
- We can predict performance gains
- We directly measure what we care about

⇒ We can confidently scale!

Critical Assumption

Train and test distributions must be the same

$$P_{\text{train}}(\mathbf{x}, y) = P_{\text{test}}(\mathbf{x}, y)$$

All the beautiful theory breaks down under distribution shift!

This is why i.i.d. sampling is so important.



The Fundamental Problem

In unsupervised/self-supervised learning:

We **optimise** one objective:
(next-token prediction, reconstruction,
contrastive loss)

But we **care about** different objectives:
(reasoning, generalisation to new tasks,
capabilities)

We cannot directly measure what we care
about!

Contrast with Supervised

Supervised:

Optimise: $\mathcal{L}(h(x), y)$

Care about: $\mathcal{L}(h(x), y)$ ✓

Unsupervised (e.g., LLM):

Optimise: $-\log P(x_{t+1}|x_{\leq t})$

Care about: reasoning, facts, safety, ...

We need a different theoretical lens.



The Key Idea

The mathematical language of **compression** provides the right framework for unsupervised learning.

Consider datasets X and Y , and a compressor C :

A good compressor uses patterns in X to help compress Y :

$$|C(\underbrace{X \circ Y}_{\text{concat}})| < |C(X)| + |C(Y)|$$

Algorithmic Mutual Information

The “gap”:

$$\underbrace{|C(X)| + |C(Y)|}_{\text{compress separately}} - \underbrace{|C(X \circ Y)|}_{\text{compress jointly}}$$

= **shared structure** between X and Y

= algorithmic mutual information

Joint compression automatically extracts shared structure!



Compression Regret

Consider algorithm A that compresses Y given access to unlabelled data X .

Regret:

$$\underbrace{|A(Y|X)|}_{\text{bits our algorithm uses}} - \underbrace{|C^*(Y|X)|}_{\text{bits optimal compressor uses}}$$

Low regret $\Leftrightarrow$ we extracted all “value” from X for predicting Y

The Key Question

What is the optimal compressor C^* ?

How do we know our neural network achieves low regret?



Kolmogorov Complexity



Definition: Kolmogorov Complexity

$$K(X) = \min_p \{|p| : U(p) = X\}$$

The length of the **shortest program** p that outputs X on universal Turing machine U .

Theorem: Optimality

For *any* compressor C and *any* data X :

$$K(X) \leq \underbrace{|C(X)|}_{\text{compressed output}} + \underbrace{K(C)}_{\text{bits to describe the compressor}} + O(1)$$

Proof: Simulation. The shortest program can implement any compressor and run it.

Interpretation

K is the **ultimate compressor**:

- Captures all algorithmic regularities
- Can simulate any other compressor
- Defines “true” information content

But: $K(X)$ is **uncomputable!**

(Requires searching over all programs)



The 'Shared Structure' Argument

Let X be training data (e.g., internet text) and Y be a downstream task (e.g., reasoning, coding).

Claim: If compressing X well requires learning patterns that also appear in Y , then training on X implicitly learns to solve Y .

The better we compress X , the more shared structure we've captured.

This is an assumption about the world — it's why we believe pretraining works.

Example: Language Models

To predict the next token well, you must learn:

- Grammar and syntax
- Facts about the world
- Logical relationships
- Causal reasoning

These patterns transfer to downstream tasks *because the same structure appears in both*.

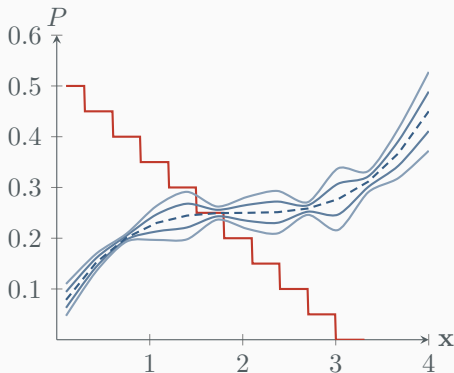
No free lunch: If Y has no shared structure with X , pretraining won't help.

Definition: no free lunch theorem

We can use methods such as cross validation to empirically choose the best method for our particular problem. However, there is no universally best model — this is sometimes called the no free lunch theorem [3].

Definition: Occam's razor

'Prefer the simplest hypothesis $\mathcal{H}$ that fits the data.' In the case of deep learning, this implies the smoothest function that fits the data.





How do we approximate K ?

SGD over neural networks $\approx$ efficient search over programs!

Architecture	$\leftrightarrow$	Program space
Weights θ	$\leftrightarrow$	Specific circuit
SGD	$\leftrightarrow$	Search algorithm

This is why architecture design is hard: SGD already searches so effectively over weight space that you need significant inductive bias gains (Transformers $\gg$ RNNs).

Maximum Likelihood = Compression

For dataset $\mathcal{D} = \{x_1, \dots, x_N\}$:

$$-\underbrace{\sum_i \log P(x_i|\theta)}_{\text{bits to encode data (cross-entropy)}} + \underbrace{|\theta|}_{\text{bits for model}}$$

Joint compression $\approx$ maximum likelihood
(if we don't overfit)

Key: Training on more data $\Rightarrow$ closer to unattainable Kolmogorov optimum $\Rightarrow$ lower regret.

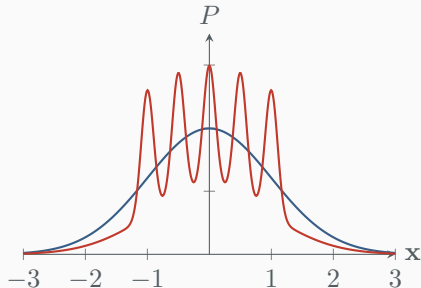


Example: problematic densities

In a more complex setting, we may have a density such as the 'Bart Simpson' density, as Nando de Freitas likes to call it

$$p(x) = \frac{1}{2}\phi(x; 0, 1) + \frac{1}{10} \sum_{j=0}^4 \phi(x; (j/2) - 1, 1/10)$$

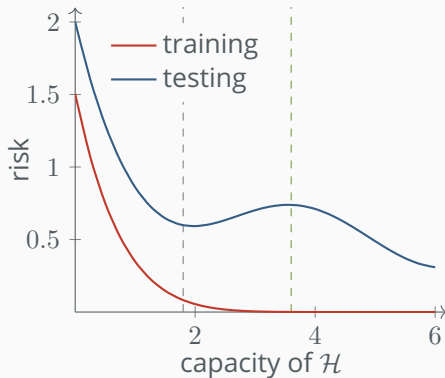
where ϕ is the normal density with mean μ and standard deviation σ . This density cannot be sufficiently estimated with a normal distribution, as the result is over-smoothed (blue).



Definition: double descent

Traditionally, we know that increasing the parameters lowers the bias (fitting), but the variance (test risk) will eventually reach a 'sweet spot' (first dashed line) and start to increase again.

The full story has a double descent curve [4], as higher capacity functions past the interpolation threshold (second dashed line) lead again to smoother fitting (Occam's razor).

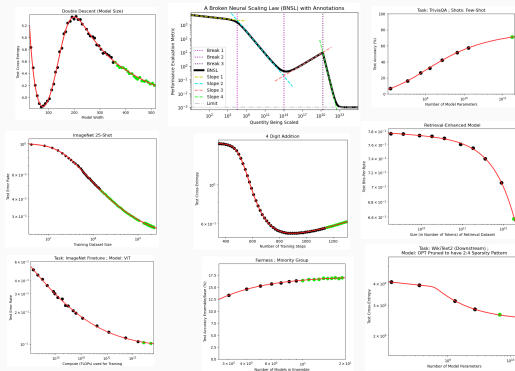


Neural scaling laws

Can we predict all this?

- Yes, yes and yes! Extremely well! (Images, LLMs, RL, Bias, ...)

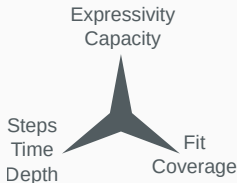
Read David Krueger's "Broken neural scaling laws" [5], ICLR 2023. Extrapolates many different criteria when considering this behaviour. See also his unifying grokking and double descent paper [6].



BNSL predictions (extrapolations) [5]

The trilemma

- The five main modelling approaches $p(\mathbf{x}) \approx ?$
- Each vary in
 - Mode coverage
 - Expressivity
 - Time or depth



Review

Method	Train Speed	Sample Speed	Num. Params.	Resolution Scaling	Free-form Jacobian	Exact Density	FID	NLL (in BPD)
Generative Adversarial Networks								
DCGAN [182]	*****	*****	*****	*****	✓	✗	37.11	-
ProGAN [114]	*****	*****	*****	*****	✓	✗	15.52	-
BigGAN [19]	*****	*****	*****	*****	✓	✗	14.73	-
StyleGAN2 + ADA [115]	*****	*****	*****	*****	✓	✗	2.42	-
Energy Based Models								
IGEBM [46]	*****	*****	*****	*****	✓	✗	37.9	-
Denoising Diffusion [87]	*****	*****	*****	*****	✓	(✓)	3.17	≤ 3.75
DDPM++ Continuous [206]	*****	*****	*****	*****	✓	(✓)	2.20	-
Flow Contrastive (EBM) [55]	*****	*****	*****	*****	✓	✗	37.30	≈ 3.27
VAEBM [247]	*****	*****	*****	*****	✓	✗	12.19	-
Variational Autoencoders								
Convolutional VAE [123]	*****	*****	*****	*****	✓	(✓)	106.37	≤ 4.54
Variational Lossy AE [29]	*****	*****	*****	*****	✗	(✓)	-	2.95
VQ-VAE [184], [235]	*****	*****	*****	*****	✗	(✓)	-	4.67
VD-VAE [31]	*****	*****	*****	*****	✓	(✓)	-	≤ 2.87
Autoregressive Models								
PixelRNN [234]	*****	*****	*****	*****	✗	✓	-	3.00
Gated PixelCNN [233]	*****	*****	*****	*****	✗	✓	65.93	3.03
PixelQNN [173]	*****	*****	*****	*****	✗	✓	49.46	-
Sparse Trans. + DistAug [32], [110]	*****	*****	*****	*****	✗	✓	14.74	2.66
Normalizing Flows								
RealNVP [43]	*****	*****	*****	*****	✗	✓	-	3.49
GLOW [124]	*****	*****	*****	*****	✗	✓	45.99	3.35
FFJORD [62]	*****	*****	*****	*****	✓	(✓)	-	3.40
Residual Flow [26]	*****	*****	*****	*****	✓	(✓)	46.37	3.28

Bond-Taylor, Willcocks et al., "Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models" in IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI) [7]

Summary

$$p(\mathbf{x}) = \prod_{i=1}^N p(x_i | x_1, \dots, x_{i-1})$$

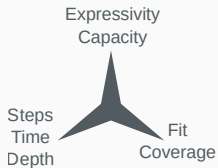
$$p(\mathbf{x}) \approx \frac{e^{-E(\mathbf{x})}}{\int_{\tilde{\mathbf{x}} \in \mathcal{X}} e^{-E(\tilde{\mathbf{x}})}} \quad \text{e.g. } \nabla_{\mathbf{x}} \log p(\mathbf{x})$$

$$\log p(\mathbf{x}) \geq \mathcal{L}_{\text{recon}}^{\text{pixel}} - D_{\text{KL}}[q_{\phi}(\mathbf{z}|\mathbf{x})||p(\mathbf{z})]$$

$$\log p(\mathbf{x}) \neq \log D(\mathbf{x}) \quad (\text{in GAN})$$

$$p(\mathbf{x}) = p_{\mathbf{z}}(f^{-1}(\mathbf{x})) \left| \det \left(\frac{\partial f^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$

See our hybrids “Unleashing transformers” [8] [↗](#) (ECCV22) & “Megapixel image generation” [9].





When measures become targets...

Current models excel on **benchmarks**, but real-world economic impact lags behind.

Why?

- Competitive pressure $\Rightarrow$ RLHF optimised for evals
- Evals $\neq$ real-world generalisation
- Goodhart's Law in action
 - *...it ceases to become a good measure*

The compression view suggests: focus on **low-regret compression** of diverse real-world data, not benchmark scores.

Why Are Humans So Capable?

Much more happens in biological neurons than typical ML models assume.

Why are humans different?

- **Continual learning** vs pretraining
- **Robust (evolved) value functions** vs narrow target benchmarks

Humans compress and adapt to an ever-evolving data stream—*like when learning to drive we optimise many values (evolved priors) to not die!*

Beyond Centralised Scaling

Learning $P(X, Y, Z, \dots)$ modalities in the context of a **temporal hypergraph** of multimodal datasets:

- Varying **availability** and **integrity**
- Different dimensions, sparsity, modalities
- Heterogeneous **topology/reachability**
- **Evolving** over time

Not all data in one place, not all data trusted equally.

Open Research Questions

- How to generalise and communicate efficiently in distributed settings?
- What are robust value functions for agents?
- How do we measure and predict scalability?
- What are the right compression/regret notions?

No longer just centralised AI scaling!

Towards “democratic” learning across heterogeneous, evolving data sources.

Summary

- **UAT:** Neural networks can approximate any function (width or depth)
- **ERM:** We minimise empirical risk as proxy for true risk
- **Statistical Learning Theory:** VC/Rademacher bounds guarantee generalisation for supervised learning *when train $\approx$ test*
- **Unsupervised is harder:** We optimise proxies for what we actually care about
- **Compression theory:** Kolmogorov complexity provides the right lens — low regret = extracting all value from data
- **SGD $\approx$ program search:** More data $\rightarrow$ approaching optimal compression $\rightarrow$ lower regret
- **Scaling laws:** We can predict generalisation empirically
- **Future:** Bridging eval-reality gap, aligned value functions, continuous decentralised learning



- [1] Patrick Kidger and Terry Lyons. **“Universal approximation with deep narrow networks”**. In: Conference on Learning Theory. 2020, pp. 2306–2327.
- [2] Richard Zhang et al. **“The unreasonable effectiveness of deep features as a perceptual metric”**. In: Proceedings of the IEEE conference on computer vision and pattern recognition. 2018, pp. 586–595.
- [3] Kevin P Murphy. **Machine learning: a probabilistic perspective**. MIT press, 2012.
- [4] Mikhail Belkin et al. **“Reconciling modern machine-learning practice and the classical bias-variance trade-off”**. In: Proceedings of the National Academy of Sciences 116.32 (2019), pp. 15849–15854.
- [5] Ethan Caballero et al. **“Broken Neural Scaling Laws”**. In: The Eleventh International Conference on Learning Representations. 2023.
- [6] Xander Davies, Lauro Langosco, and David Krueger. **“Unifying Grokking and Double Descent”**. In: arXiv preprint arXiv:2303.06173 (2023).



- [7] Sam Bond-Taylor et al. **“Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models”**. In: IEEE Transactions on Pattern Analysis and Machine Intelligence (2021), pp. 1–1. DOI: 10.1109/TPAMI.2021.3116668.
- [8] Sam Bond-Taylor et al. **“Unleashing Transformers: Parallel Token Prediction with Discrete Absorbing Diffusion for Fast High-Resolution Image Generation from Vector-Quantized Codes”**. In: European Conference on Computer Vision (ECCV) (2022). DOI: 10.1007/978-3-031-20050-2_11.
- [9] Alex F McKinney and Chris G Willcocks. **“Megapixel Image Generation with Step-Unrolled Denoising Autoencoders”**. In: arXiv preprint arXiv:2206.12351 (2022).